

CXR OpenTelemetry Lab Manual

SW.11 — Starter milestone #7

Connecting rehearsal :8251 to Jaeger

CXR Ops Lab — `cxr-ops-lab`

2026-05-31

Abstract

Step-by-step lab for **OpenTelemetry (SW.11)**: how **cxr-ui-rehearsal** on **:8251** sends traces to the OTel Collector and how **Jaeger UI (:16686)** shows them when you click **Run Analysis** in Claim Studio. The collector has **no web UI**; Jaeger is the viewer. Use this PDF as the implementation record for traces vs metrics (Prometheus/Grafana).

Contents

1	Where you view telemetry	2
2	How :8251 connects to Jaeger (highlight)	3
2.1	End-to-end path	3
2.2	What “connects” the rehearsal app	3
2.3	Why Run Analysis appeared in Jaeger immediately	3
2.4	GET vs POST traces (why the list is “divided”)	4
2.5	systemd vs manual dev (:8251)	4
3	Step-by-step procedure	5
3.1	Step 1 — Start observe + OTel stack	5
3.2	Step 2 — Install npm packages (once)	5
3.3	Step 3 — Wire :8251 (env + dev server)	5
3.4	Step 4 — Golden path: Run Analysis → Jaeger	5
3.5	Step 5 — Write evidence note	5
4	Implementation checklist (repeat on another machine)	6
5	Optional: Compose :3000	7
6	File reference	8
7	Troubleshooting	9
8	Study plan linkage	10
9	Live adjustments (made traces work on :8251)	11
10	UI evidence — Jaeger (:16686)	12

Chapter 1

Where you view telemetry

Signal	Tool	URL	Web UI?
Metrics	Prometheus	:9090	Yes
Dashboards	Grafana	:3001	Yes
Traces	Jaeger	:16686	Yes — use this
OTLP ingest	OTel Collector	:4318	No (API only)

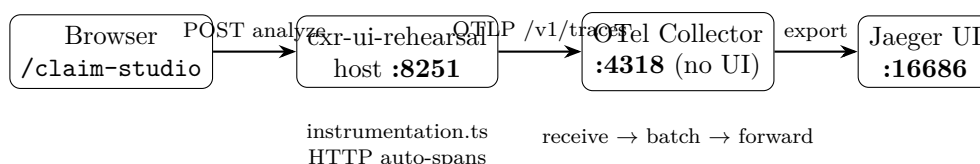
Key point: Opening <http://localhost:4318> in a browser often returns 404 — that is normal. The collector runs in the background. You only *see* trace data in **Jaeger**.

Chapter 2

How :8251 connects to Jaeger (highlight)

This chapter documents the connection you verified: **Run Analysis** on Claim Studio and traces appearing in Jaeger within seconds.

2.1 End-to-end path



2.2 What “connects” the rehearsal app

1. **Observe stack running** — `./scripts/07-observe-up.sh` starts collector + Jaeger on Docker network `cyr_observe`.
2. **Environment variables on the :8251 process** (must be set *before next dev*):

```
export OTEL_EXPORTER_OTLP_ENDPOINT=http://127.0.0.1:4318
export OTEL_SERVICE_NAME=cyr-ui-rehearsal
```

Template: `cyr-ops-lab/.env.otel.example`.

3. **Next.js instrumentation hook** — `instrumentation.ts` calls `register()` at Node process start; loads `instrumentation/register-otel-node.ts` (NodeSDK + OTLP HTTP exporter + HTTP auto-instrumentation). If `OTEL_EXPORTER_OTLP_ENDPOINT` is unset, tracing is off (normal systemd :8251).
4. **User action** — **Run Analysis** sends `POST /api/claim-studio/analyze`. OTel records spans on the Node server and exports them to the collector.
5. **Jaeger UI** — Search → Service `cyr-ui-rehearsal` → new **POST** trace (often 5–11s duration).

2.3 Why Run Analysis appeared in Jaeger immediately

- The browser POST hits the same Next.js process you started with `OTEL_*` set.

- HTTP instrumentation creates a trace for that request (not for the whole browser session).
- Spans are sent to `http://127.0.0.1:4318/v1/traces`, then forwarded to Jaeger.
- Jaeger Search lists the trace under service `cxr-ui-rehearsal`.
- Waterfall typically shows `executing api route` for most of the duration (Python analyzer subprocess is *not* a separate span in this bootcamp scope).

2.4 GET vs POST traces (why the list is “divided”)

HTTP	When	Typical duration
GET	Page loads, navigation	ms – 2 s
POST	Run Analysis	~5–11 s

Jaeger shows **one trace per HTTP request**. Several GETs plus one or more POSTs = multiple rows in Search.

2.5 systemd vs manual dev (:8251)

The user unit `cxr-rehearsal-dev` serves daily dev on :8251 **without** `OTEL_*`. For SW.11:

```
systemctl --user stop cxr-rehearsal-dev
export OTEL_EXPORTER_OTLP_ENDPOINT=http://127.0.0.1:4318
export OTEL_SERVICE_NAME=cxr-ui-rehearsal
cd cxr-ui-prune-rehearsal/cxr-ui
npm run dev:rehearsal
```

After the lab: `systemctl -user start cxr-rehearsal-dev` for normal untraced dev.

Chapter 3

Step-by-step procedure

3.1 Step 1 — Start observe + OTel stack

```
cd cxr-ops-lab
./scripts/07-observe-up.sh
./scripts/11-otel-smoke.sh
```

Open <http://localhost:16686> (Jaeger UI only).

3.2 Step 2 — Install npm packages (once)

```
cd cxr-ui-prune-rehearsal/cxr-ui
npm install
```

3.3 Step 3 — Wire :8251 (env + dev server)

Stop conflicting :8251 service; export OTel env; start rehearsal dev (see Chapter 2).

3.4 Step 4 — Golden path: Run Analysis → Jaeger

1. <http://localhost:8251/claim-studio>
2. Click **Run Analysis**; confirm POST `/api/claim-studio/analyze` returns 200.
3. <http://localhost:16686> → Search → Service `cxr-ui-rehearsal` → Find Traces.
4. Open newest **POST** trace; screenshot search + waterfall for evidence.

Success criterion: POST trace visible right after Run Analysis — that proves :8251 is connected to Jaeger.

3.5 Step 5 — Write evidence note

Edit `evidence/SW11-otel-verify-2026-05-29.md`:

- Document :8251 + OTEL_* → :4318 → Jaeger connection.
- Spans start: `instrumentation.ts`, HTTP auto-instrumentation.
- Not traced yet: Python `spawn` subprocess.
- Attach PNGs (e.g. `SW11-jaeger-search-*.png`, `SW11-jaeger-waterfall-post-analyze-*.png`).

Chapter 4

Implementation checklist (repeat on another machine)

1. Start collector + Jaeger (`07-observe-up.sh`).
2. Set `OTEL_EXPORTER_OTLP_ENDPOINT` and `OTEL_SERVICE_NAME` before starting Next.
3. Ensure `instrumentation.ts` + `register-otel-node.ts` in the app; `next.config.ts` lists OTel in `serverExternalPackages`.
4. Generate traffic (Run Analysis).
5. Verify only in Jaeger :16686 (not :4318).

Chapter 5

Optional: Compose :3000

Rebuild `cxr-ui:compose` after OTel packages are in the image context, then:

```
./scripts/07-observe-up.sh
docker compose -f compose.yaml -f compose.host.yaml \
  -f compose.otel-link.yaml up -d
```

Jaeger service name: `cxr-ui-compose`.

Chapter 6

File reference

File	Purpose
compose.observe.yaml	Prometheus, Grafana, Jaeger, collector
observe/otel-collector-config.yaml	OTEL Jaeger out
instrumentation.ts	Next.js register hook (edge-safe entry)
instrumentation/registerNodeSDK.ts	Node SDK + shutdown handlers
.env.otel.example	Env template for :8251 / :3000
compose.otel-link.yaml	Compose network link to collector
scripts/11-otel-smoke.sh	Health probe

Chapter 7

Troubleshooting

- **No traces** — OTel env must be set *before next dev*; restart dev after exporting vars.
- **Analyze works, Jaeger empty** — Filter Service = `cxr-ui-rehearsal`; generate traffic after server start.
- **:4318 browser 404** — Expected; use Jaeger :16686.
- **Port 8251 busy** — Stop `cxr-rehearsal-dev` or old dev process.
- **Disable OTel** — `export OTEL_SDK_DISABLED=true`.

Chapter 8

Study plan linkage

Item	Maps to
Starter milestone #7	SW.11
Module 1 step M1.6	OTel concepts + C4 telemetry edge
After this lab	SW.12–SW.18 electives

Chapter 9

Live adjustments (made traces work on :8251)

- **Next.js 16 searchParams** — `app/page.tsx`: `await Promise.resolve(searchParams)` (fixes runtime error on `/`).
- **Instrumentation split** — `instrumentation.ts` = thin Edge-safe hook; `instrumentation/register-on-exit.ts` = NodeSDK, OTLP exporter, `process.on` shutdown (avoids Edge warnings).
- **Resource API** — `new Resource({...})` instead of deprecated `resourceFromAttributes`.
- **next.config.ts** — `serverExternalPackages` lists OTel packages for Next bundler.
- **:8251 ownership** — `systemctl -user stop cxx-rehearsal-dev` before `OTEL_*+npm run dev:rehearsal`; `env` — *expected; only Jaeger : 16686 is the trace UI*.

Chapter 10

UI evidence — Jaeger (:16686)

Verified after Claim Studio **Run Analysis** on **:8251** with `OTEL_EXPORTER_OTLP_ENDPOINT=http://127.0.0.1`

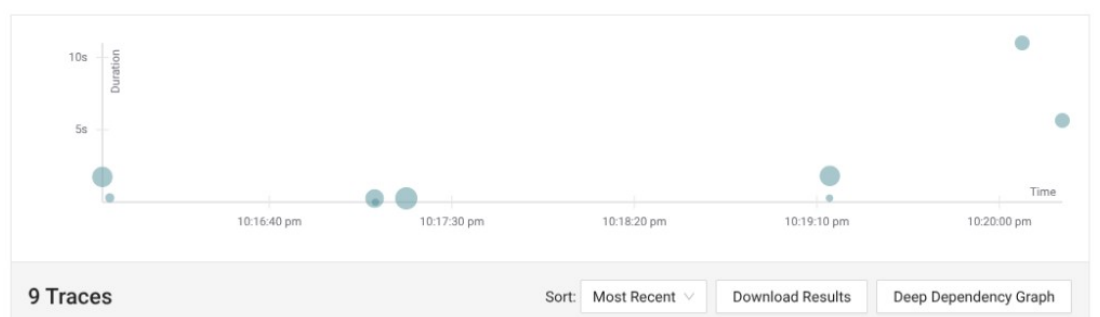


Figure 10.1: Jaeger Search — service `cxr-ui-rehearsal`; POST `analyze` traces visible.

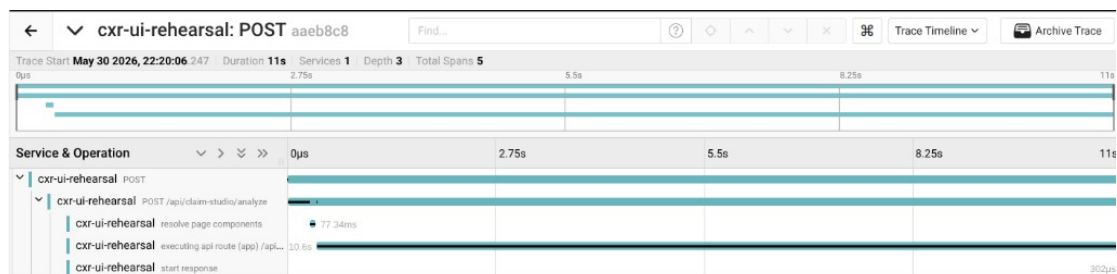


Figure 10.2: Waterfall for POST `/api/claim-studio/analyze` ($\sim 5\text{--}11\text{ s}$).